

The Rest of Rust: A Beginner's Gateway



Explore what Rust can do and find out that it is not as tough to learn as is made out to be...

The urge to replace C for systems programming is stronger than ever due to the serious kinds of bugs that arise from its design. Although several alternatives have been proposed, Rust seems to be the only promising one so far, welcomed even by Linus Torvalds, notorious for his hostility towards any language other than C for kernel development.

However, it is widely observed that the adoption rate of Rust does not match the excitement for it. The primary reason cited is the steep learning curve. But is that the whole story?

Being memory safe with the same performance as of C and C++ is a primary goal of Rust. Naturally, almost all learning material on Rust out there has a heavy focus on the underlying memory model. Unfortunately, this can easily ward off beginners. But

what if we start forgetting memory management for a while? Yes, one has to start facing the borrow checker at some point, but there's a lot a beginner can do without it. That includes features that are comparable to the ones that we're already familiar with in other programming languages. An encouraging entry like this would make the real deal more enjoyable.

Getting Rust

Setting up a Rust environment depends on the platform you use, and everything you need to know and get is available on rust-lang.org. But for the purpose of this article, the simple online playground available at play.rust-lang.org will suffice.

By the way, I recommend you *not* install Rust from your distro's repository, especially if it prefers stability over being up-to-date (like Debian). Rust is still evolving, and

there is no guarantee that you'll get even these simple examples working with your distro's version of Rust.

Enums that are unions

People who are familiar with functional programming languages won't need any introduction to an enums in Rust. They'll instantly get it and love it. However, a C programmer, even if they manage to get it, may find it unnecessarily overloaded. Let's demystify Rust enums.

Have a look at the following sample program:

```
enum Vehicle {
    Bicycle,
    Car(String),
}

fn print_vehicle(v:Vehicle) {
    match v {
        Vehicle::Bicycle =>
            println!("A bicycle."),

        Vehicle::Car(regno) =>
            println!("A car with reg. no.:
    {}.", regno),
    };
}

fn main() {
    let v1 = Vehicle::Car("KL 00 X 0000".
to_string());
    let v2 = Vehicle::Bicycle;
    let v3 = Vehicle::Car("KL 00 Y 1111".
to_string());

    print_vehicle(v1);
    print_vehicle(v2);
    print_vehicle(v3);
}
```